

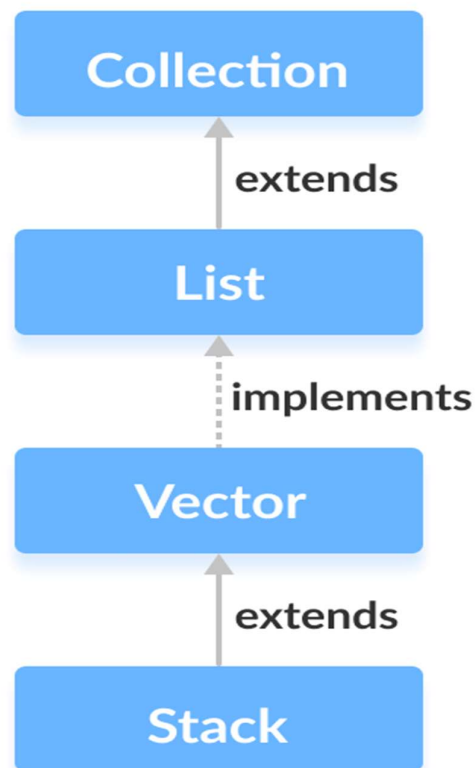
Interfaces in Collection Framework

1. Stack Interface :-

+ What is Stack :-

- Stack in Java is **not an interface**, but a **class** part of the **java.util** package.
- Stack of **collection** that follows the **LIFO (Last-In, First-Out)** principle meaning the last element **added is the first one to be removed**.

+ Hierarchy of Stack :-



Methods in Stack :-

Method	Description
push(E item)	Adds an item to the top of the stack
pop()	Removes and returns the top element
peek()	Returns the top element without removing
empty()	Checks if the stack is empty

Syntax of stack :-

Stack<Integer> stack = new Stack<>();

Key Points :-

- **Order** :- Follows LIFO (Last In, First Out)
- **Duplicate elements** :- Allowed
- **Null elements** :- Allowed
- **Synchronization** :- Thread-safe (because Stack extends Vector)
- **Insertion/Removal** :- Done using push() and pop()
- **Access** :- Only the top element can be accessed directly
- **Performance** :- Slower than ArrayDeque (preferred for stack operations)

Real Life of Stack :-

- You **add** a new plate on top (**push**)
- You **remove** the top plate first (**pop**)
- You can **look** at the top plate (**peek**)

Implementation of Stack :-

Input :-

```
1 package Collection_Framework;
2
3 import java.util.Stack;
4
5 public class stack_implementation {
6
7     public static void main(String[] args) {
8
9         Stack s = new Stack();
10        // Used to add element in stack :- LIFO
11        s.push(34);
12        s.push("Nikhil");
13        s.push(9.8);
14        s.push(true);
15        s.push("#");
16
17        System.out.println("Stack is : "+s);
18
19        // Used to remove object from the stack
20        s.pop();
21        System.out.println("After pop stack is : "+s);
22
23        // Used to remove random object
24        s.remove(1);
25        System.out.println("After remove stack is : "+s);
26
27        System.out.println("Stack Capacity : "+s.capacity());
28
29        System.out.println("After peek stack is : "+s.peek());
30
31        System.out.println("After Search : "+s.search(34));
32
33    }
34 }
```

Output :-

```
Stack is : [34, Nikhil, 9.8, true, #]
After pop stack is : [34, Nikhil, 9.8, true]
After remove stack is : [34, 9.8, true]
Stack Capacity : 10
After peek stack is : true
After Search : 3
```

Input :-

```
1 package Collection_Framework;
2 import java.util.Scanner;
3 import java.util.Stack;
4
5 public class stack_bank {
6
7     public static void main(String[] args) {
8
9         Scanner s = new Scanner(System.in);
10
11         Stack<String> bank = new Stack<String>();
12         System.out.println("Enter the size of stack");
13         int size = s.nextInt();
14
15
16         System.out.println("Enter the bank names : ");
17         for(int i=0;i<size;i++)
18         {
19             String bname = s.next();
20             bank.push(bname);
21         }
22
23         System.out.println("Before removing Bank collection is : "+bank);
24         // Check bank SBI
25         for(int i=0;i<size;i++)
26         {
27             String b=bank.get(i);
28             if(b.equalsIgnoreCase("SBI"))
29             {
30                 bank.remove("SBI"); // Removing SBI bank
31             }
32         }
33         System.out.println("After removing Bank collection is : "+bank);
34     }
35 }
```

Output :-

Enter the size of stack

2

Enter the bank names :

HDFC

SBI

Before removing Bank collection is : [HDFC, SBI]

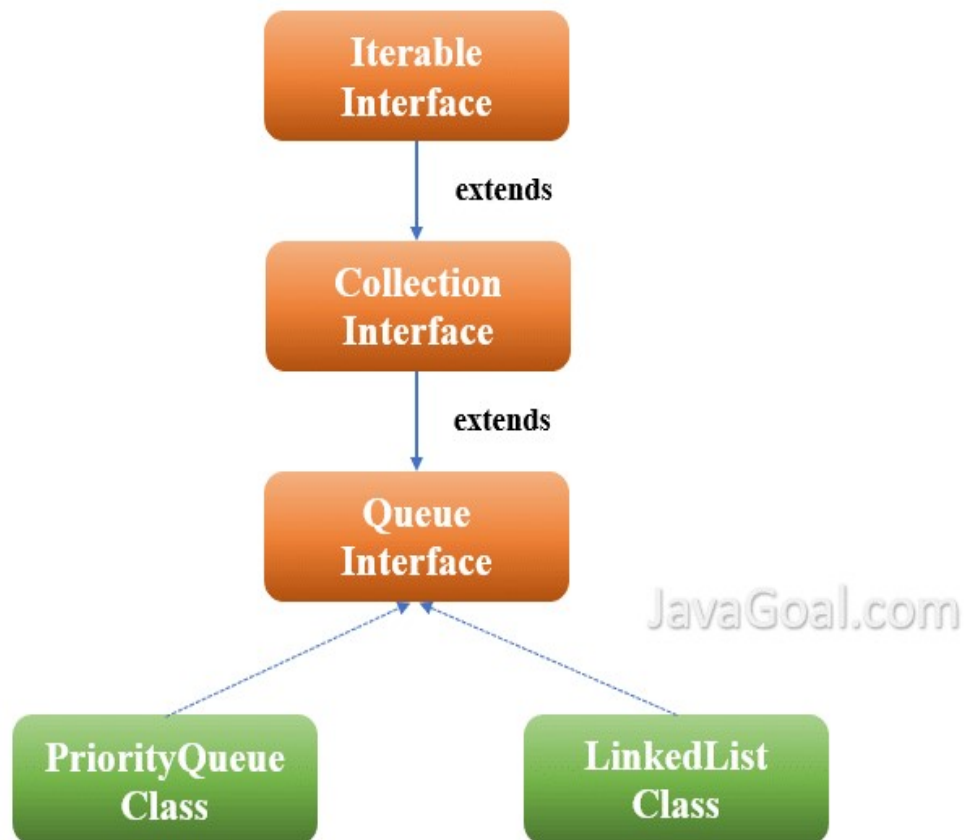
After removing Bank collection is : [HDFC]

2. Queue Interface :-

📌 What is Queue :-

- The Queue Interface is (part of **java.util package**) is can store elements in a **FIFO (First-In, First-Out)** order.
- It represents a **collection designed for holding elements prior to processing** like a line of people waiting for a service.

📌 Hierarchy of Queue Interface :-



Methods in Queue :-

Method	Description
add(E e)	Inserts an element
offer(E e)	Inserts an element
remove()	Removes and returns the head
poll()	Removes and returns the head
element()	Retrieves, but doesn't remove, the head
peek()	Retrieves, but doesn't remove, the head

Syntax of Queue :-

Queue<Integer> queue = new LinkedList<>();

Key Points :-

- **Order** :- Follows FIFO (First In, First Out)
- **Null elements** :- Not allowed in most implementations (PriorityQueue, ArrayDeque)
- **Duplicates** :- Allowed
- **Synchronization** :- Not synchronized (can be synchronized manually)
- **Common Implementations** :- LinkedList, PriorityQueue, ArrayDeque
- **Insertion** :- offer() or add()
- **Removal** :- poll() or remove()
- **Access front element** :- peek() or element()

Implementation of Queue :-

Input :-

```
1 package Collection_Framework;
2
3 import java.util.ArrayDeque;
4 import java.util.LinkedList;
5 import java.util.Queue;
6
7 public class Queue_implementation {
8
9     public static void main(String[] args) {
10
11         Queue q = new ArrayDeque();
12         q.add("Nikhil");
13         q.add(800);
14         q.add("Pune");
15         q.add(564534.8);
16         System.out.println("Queue is : "+q);
17
18         System.out.println("Head object of queue : "+q.element());
19
20         q.offer('%');
21         System.out.println("After queue is : "+q);
22
23         System.out.println("Peek : "+q.peek());
24         System.out.println("After Peek is : "+q);
25
26         System.out.println("Poll : "+q.poll());
27         System.out.println("After Poll is : "+q);
28     }
29 }
```

Output :-

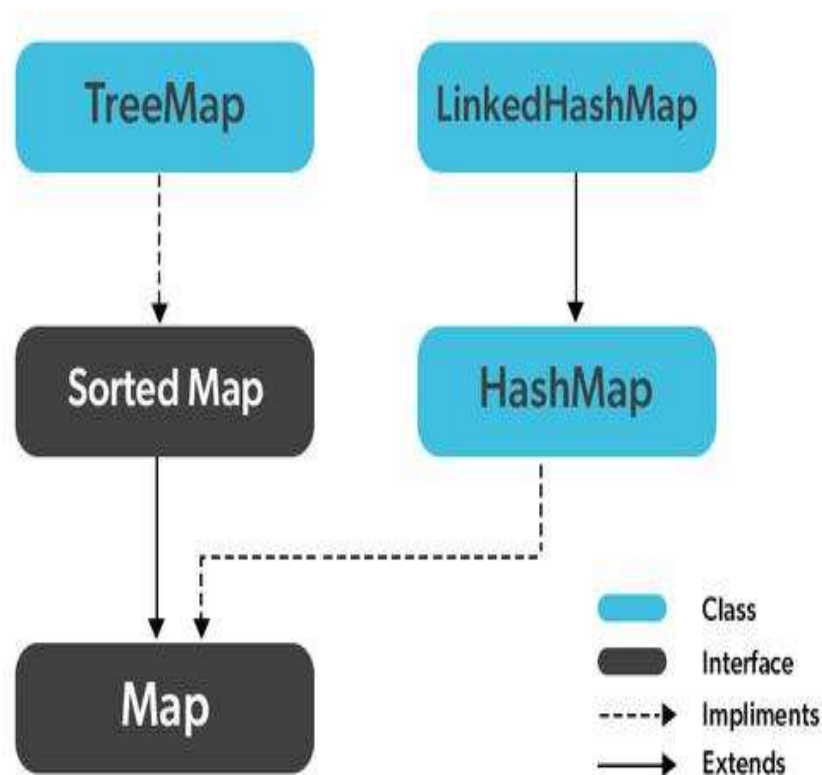
```
Queue is : [Nikhil, 800, Pune, 564534.8]
Head object of queue : Nikhil
After queue is : [Nikhil, 800, Pune, 564534.8, %]
Peek : Nikhil
After Peek is : [Nikhil, 800, Pune, 564534.8, %]
Poll : Nikhil
After Poll is : [800, Pune, 564534.8, %]
```

3. Map Interface :-

+ What is Map Interface :-

- The Map interface in Java is a core component of the Java **Collections Framework**, **designed to store** data in key-value pairs.
- It differs from other collection types like **List** and **Set** because it does not directly extend the Collection interface.

+ Hierarchy of Map Interface :-



Methods in Map Interface :-

Method	Description
put(K key, V value)	Inserts key-value pair
get(Object key)	Returns value for the given key
remove(Object key)	Removes entry for the given key
containsKey(Object key)	Checks if key exists
containsValue(Object value)	Checks if value exists
keySet()	Returns all keys as a Set
values()	Returns all values as a Collection
entrySet()	Returns all key-value pairs as a Set of Map.Entry
size()	Returns total number of entries
clear()	Removes all entries

Syntax of Map Interface :-

```
Map<Integer, String> map = new HashMap<>();
```

Key Points :-

- **Stores** :- Data as key value pairs
- **Order** :- Depends on implementation
- **Duplicate Keys** :- Not allowed
- **Duplicate Values** :- Allowed

Implementation of Map Interface :-

Input :-

```
1 package Collection_Framework;
2
3 import java.util.HashMap;
4
5 public class Hash_Map {
6
7     public static void main(String[] args) {
8
9         // Nongeneric Collection
10
11         HashMap h = new HashMap();
12         h.put(101, "Nikhil");
13         h.put(102, 'S');
14         h.put(103, 99876543451);
15
16         System.out.println("Nongeneric collection of HashMap : "+h);
17
18         System.out.println("Contains Value : "+h.containsValue('S'));
19
20         System.out.println("Contains Key : "+h.containsKey(103));
21
22         System.out.println("Key set : "+h.keySet());
23
24         System.out.println("Remove Key : "+h.remove(101));
25     }
26 }
```

Output :-

```
Nongeneric collection of HashMap : {101=Nikhil, 102=S, 103=9987654345}
Contains Value : true
Contains Key : true
Key set : [101, 102, 103]
Remove Key : Nikhil
```

Input :-

```
1 package Collection_Framework;
2
3 import java.util.HashMap;
4
5 public class HashMap_Generic {
6
7     public static void main(String[] args) {
8
9         // Generic Collection
10
11         HashMap<Integer,String> emp = new HashMap<Integer,String>();
12         emp.put(1, "Nikhil");
13         emp.put(2, "Omkar");
14         emp.put(3, "Sahil");
15         System.out.println("Generic of hashmap is : "+emp);
16     }
17 }
```

Output :-

```
Generic of hashmap is : {1=Nikhil, 2=Omkar, 3=Sahil}
```

Input :-

```
1 package Collection_Framework;
2
3 import java.util.TreeMap;
4
5 public class Tree_Map {
6
7     public static void main(String[] args) {
8
9         TreeMap tm = new TreeMap();
10        tm.put(70, "Nikhil");
11        tm.put(50, "Omkar");
12
13        System.out.println("Non Generic Collection of treemap is : "+tm);
14    }
15 }
```

Output :-

Non Generic Collection of treemap is : {50=Omkar, 70=Nikhil}

Difference Between List , Set , Queue and Map :-

Feature	List	Set	Map
Duplicates	Allows duplicates	No duplicates allowed	Keys must be unique; values can repeat
Order	Maintains insertion order	No guaranteed order	Order depends on implementation
Null Values	Allows multiple nulls	Allows only one null	Allows one null key, many null values
Implementation Classes	ArrayList, LinkedList	HashSet, LinkedHashSet, TreeSet	HashMap, LinkedHashMap, TreeMap, Hashtable, ConcurrentHashMap
Access by Index	Supported using get()	Not supported	Not supported
When to Use	When you need ordered, index-based access	When you need unique elements	When you need key–value storage
Traversal	Using ListIterator	Using Iterator	Using keySet(), values(), or entrySet()